

\# N100 Project – Analyst User Guide

\## 1. Introduction

The N100 Project is an analytics platform designed to analyze companies, financial performance, key performance indicators, peer comparisons, and screening results.

This guide explains how an analyst can use the Streamlit screener, dashboard screens, PDF tearsheets, FastAPI endpoints, and troubleshooting procedures.

\## 2. Project Overview

The project provides an end-to-end analytics workflow:

- 1\. Data ingestion
- 2\. Data cleaning and normalization
- 3\. SQLite database storage
- 4\. Financial analytics
- 5\. KPI calculations
- 6\. Company screening
- 7\. Peer comparison
- 8\. Interactive dashboard visualization
- 9\. FastAPI API endpoints
- 10\. PDF tearsheet generation

The main purpose of the system is to help analysts explore company-level financial information and compare companies using different financial metrics.

\## 3. Starting the Application

\### 3.1 Open the Project

Open PowerShell and go to the project directory:

D:\N100_Project

The project contains the source code, database, SQL files, reports, output files, tests, and documentation.

\### 3.2 Activate the Virtual Environment

Run:

```
.\venv\Scripts\Activate.ps1
```

After successful activation, the terminal should show:

```
(venv) PS D:\N100\_Project>
```

\\### 3.3 Start the FastAPI Server

Run:

```
python -m uvicorn src.api:app --reload
```

The API server normally starts at:

```
http://127.0.0.1:8000
```

\\### 3.4 Open Swagger Documentation

Open the following address in the browser:

```
http://127.0.0.1:8000/docs
```

Swagger UI displays the available API endpoints and allows the analyst to test them directly.

\\## 4. Health Check

The health endpoint is used to verify that the API and database are working correctly.

Endpoint:

```
GET /api/v1/health
```

Example:

```
curl -X GET "http://127.0.0.1:8000/api/v1/health"
```

A successful request should return HTTP 200.

HTTP 200 means that the request was successfully processed.

\\## 5. Company List

The company endpoint provides company information stored in the project database.

Endpoint:

GET /companies

Example:

```
curl -X GET "http://127.0.0.1:8000/companies"
```

The response provides company-level information available in the database.

The analyst can use this endpoint to verify that company records are available.

6. Company Profile

The company profile screen is used to inspect information about an individual company.

Typical workflow:

- 1\ Open the company profile screen.
- 2\ Select a company.
- 3\ Review company information.
- 4\ Review available financial metrics.
- 5\ Review historical performance.
- 6\ Continue to ratio or peer-analysis screens when required.

For testing, a company ID such as ABB can be used.

7. Company Ratios

The company ratios endpoint provides historical financial-ratio information for a selected company.

Endpoint:

GET /companies/{company_id}/ratios

Example:

```
curl -X GET "http://127.0.0.1:8000/companies/ABB/ratios"
```

The response contains ratio information for the selected company.

The analyst can use this information to study changes in financial performance across different years.

8. Streamlit Screener

The Streamlit screener is used to filter companies according to selected financial and analytical conditions.

The screener can be used to:

- \- Select financial metrics
- \- Apply screening conditions
- \- Filter companies
- \- Review matching companies
- \- Download screening results
- \- Compare selected companies

Recommended Workflow

- 1\. Open the Streamlit application.
- 2\. Navigate to the Screener page.
- 3\. Select the required metric.
- 4\. Enter the required threshold or condition.
- 5\. Apply the filter.
- 6\. Review the returned companies.
- 7\. Download the results if required.

\\# 9. Screener Results

After applying the selected filters, the screener displays companies that satisfy the screening conditions.

The analyst should verify:

- \\- Number of companies returned

- \\- Company IDs

- \\- Company names

- \\- Selected metric values

- \\- Applied screening conditions

When available, the results can be downloaded as a CSV file for further analysis.

\\# 10. Dashboard Navigation

The dashboard contains multiple analytical screens that help the analyst explore company and financial information.

\\### 10.1 Home / Overview

The overview screen provides a high-level summary of the project and the available analytical sections.

\\### 10.2 Company Profile

The company profile screen focuses on an individual company.

Use this screen to review:

- \\- Company information

- \\- Financial metrics

\- Historical values

\- Key ratios

\\### 10.3 KPI Dashboard

The KPI dashboard presents important performance indicators in a visual format.

Analysts can use KPI cards and charts to identify important changes and trends.

\\### 10.4 Peer Comparison

The peer comparison screen allows a company to be compared with other companies or peer groups.

This is useful for understanding relative performance.

\\### 10.5 Screener

The screener allows analysts to filter companies using predefined financial conditions.

\\### 10.6 Analytics

Analytics screens provide calculated metrics and analytical outputs.

Use these screens to identify trends, unusual values, and relative company performance.

\\## 11. Peer Comparison

Peer comparison is useful when an analyst wants to compare a selected company against similar companies.

Recommended steps:

- 1\.. Select the target company.
- 2\.. Identify the relevant peer group.

- 3\ Review the available financial metrics.
- 4\ Compare the target company with its peers.
- 5\ Identify stronger and weaker metrics.
- 6\ Use the comparison results for further analysis.

Peer comparison should be used together with other financial metrics before making a final conclusion.

\\# 12. Outlier Analysis

Outlier analysis identifies companies whose metric values are significantly different from the rest of the dataset.

Outlier analysis can help analysts investigate:

- \- Extremely high values
- \- Extremely low values
- \- Unusual financial performance
- \- Potential data-quality issues

An identified outlier should be investigated before drawing a final business conclusion.

\\# 13. Correlation Analysis

Correlation analysis helps identify relationships between selected KPIs.

The correlation heatmap can be used to:

- 1\ Review the available metrics.
- 2\ Inspect the correlation matrix.
- 3\ Identify strongly related metrics.

4\ Identify weakly related metrics.

5\ Investigate potentially important relationships.

Correlation indicates the strength and direction of a relationship between variables. It does not by itself establish causation.

\\# 14. Generating PDF Tearsheets

A PDF tearsheet provides a compact summary of a company or analytical result.

\\### Recommended Workflow

1\ Open the required company or analysis screen.

2\ Select the required company.

3\ Review the displayed metrics.

4\ Select the PDF or tearsheet generation option.

5\ Generate the PDF.

6\ Open the generated PDF.

7\ Verify that the contents are readable.

8\ Save the PDF in the required reports or tearsheets directory.

\\### PDF Quality Check

Before using a generated PDF, verify:

\- Correct company name

\- Correct financial metrics

\- Correct charts

\- No text overflow

\- No missing sections

\- Readable formatting

\- Correct file name

\## 15. API Usage with cURL

The API can be tested from PowerShell or another terminal using cURL commands.

\### Health API

```
curl -X GET "http://127.0.0.1:8000/api/v1/health"
```

\### Companies API

```
curl -X GET "http://127.0.0.1:8000/companies"
```

\### Company Ratios API

```
curl -X GET "http://127.0.0.1:8000/companies/ABB/ratios"
```

\### API Documentation

The interactive Swagger documentation is available at:

<http://127.0.0.1:8000/docs>

Swagger allows analysts to enter endpoint parameters and execute API requests without manually writing cURL commands.

\## 16. Understanding HTTP Status Codes

Common API status codes include:

\\### 200 – Successful Response

The request was successfully processed.

A status code of 200 indicates that the API endpoint returned a successful response.

\\### 404 – Not Found

The requested resource or company may not exist.

Check the company ID and requested endpoint.

\\### 422 – Validation Error

One or more request parameters do not satisfy the required format.

Check the endpoint parameters and enter valid values.

\\### 500 – Internal Server Error

The server encountered an unexpected problem.

When a 500 error occurs, check the terminal running Uvicorn for the complete traceback.

The traceback normally identifies the Python file, function, and SQL statement that caused the error.

\\## 17. Troubleshooting

\\### Problem: API Does Not Start

Check that the virtual environment is activated.

Run:

```
.\venv\Scripts\Activate.ps1
```

Then start the server again:

```
python -m uvicorn src.api:app --reload
```

\\### Problem: Port Already in Use

Stop the existing Uvicorn process or use another available port.

Example:

```
python -m uvicorn src.api:app --reload --port 8001
```

\\### Problem: Database Error

Verify that the SQLite database exists in the expected database directory.

Also verify that the required tables and columns exist.

\\### Problem: No Such Column Error

An error such as:

```
sqlite3.OperationalError: no such column
```

usually means that the SQL query expects a column that is not present in the current database schema.

Check:

1\. sql/schema.sql

2\. Database schema

3\. Data-loading scripts

4\ SQL queries used by the API

\\### Problem: HTTP 500

Check the Uvicorn terminal for the complete traceback.

The traceback normally identifies the exact Python file, function, and SQL query that caused the problem.

\\### Problem: Swagger Returns 422

Check the endpoint parameters and make sure all required values are entered correctly.

\\### Problem: Empty Results

Verify:

\\- Company ID

\\- Database contents

\\- Applied filters

\\- Required financial records

\\- Selected year or metric

\\## 18. Testing the Project

Run the complete test suite from the project root:

```
pytest -q
```

A successful test run should show all tests passing.

The test suite should be run after major code, database, or API changes.

The test result should be recorded as part of the final project validation.

\\# 19. Database Verification

The project uses SQLite for database storage.

Before relying on API results, verify that:

- \\- Required tables exist
- \\- Required columns exist
- \\- Company records are present
- \\- Financial records are present
- \\- Database schema matches the SQL queries
- \\- No unexpected schema mismatch exists

The database verification scripts available in the project can be used for this purpose.

\\# 20. Analyst Best Practices

Analysts should follow these practices:

- 1\\. Verify the company selected before interpreting results.
- 2\\. Check the reporting period of financial metrics.
- 3\\. Compare multiple metrics instead of relying on a single KPI.
- 4\\. Investigate unusual values and outliers.
- 5\\. Verify unexpected financial values.
- 6\\. Check API status codes when using the API.
- 7\\. Validate exported CSV files.

8\ Check generated PDF tearsheets before sharing them.

9\ Keep downloaded reports organized.

10\ Record important analytical observations.

21. Recommended Analysis Workflow

A typical analyst workflow is:

Open Dashboard

 ↓

Select Company

 ↓

Review Company Profile

 ↓

Review KPIs

 ↓

Review Financial Ratios

 ↓

Compare with Peers

 ↓

Run Screener

 ↓

Investigate Outliers

 ↓

Review Correlations

 ↓

Generate PDF Tearsheets

 ↓

Export Required Results

 ↓

Validate Final Results

This workflow helps maintain a consistent approach to company analysis.

\\# 22. Final Checklist

Before completing an analysis, verify:

\\- \\[] Correct company selected

\\- \\[] Correct reporting period selected

\\- \\[] KPIs reviewed

\\- \\[] Financial ratios reviewed

\\- \\[] Peer comparison reviewed

\\- \\[] Screener conditions verified

\\- \\[] Outliers investigated

\\- \\[] API results checked where applicable

\\- \\[] CSV export checked

\\- \\[] PDF tearsheet checked

\\- \\[] No text overflow in PDF

\\- \\[] Results saved in the correct directory

\\# 23. Documentation and Support

For technical problems, first check the application terminal, API response code, database schema, and project test results.

When an API request fails:

- 1\\. Check the HTTP status code.
- 2\\. Check the Uvicorn terminal.
- 3\\. Read the traceback carefully.
- 4\\. Identify the affected Python function.
- 5\\. Check the related SQL query.
- 6\\. Verify the database schema.
- 7\\. Run the test suite after making a correction.

\\# 24. Conclusion

The N100 Project provides an integrated environment for company analysis, financial-ratio analysis, KPI analysis, screening, peer comparison, visualization, and API-based access.

Analysts should use the dashboard together with the API and exported reports to validate important results and maintain a consistent analytical workflow.

The analyst should always verify important results before using them for business or investment-related decisions.